

Quant Research Platform — Platform Documentation

Module contracts, data schemas and data flow of the desktop app. Updated 2026-10-01. Generated: edit `docs/app_documentation/APP_DOCUMENTATION.md`, then run `python docs/app_documentation/build_app_doc.py` to rebuild `Quant_app_documentation.pdf`.

The strategy-author reference (every detail of writing a strategy) is the separate `STRATEGY_GUIDE.md` / `Strategy_Guide.pdf`; the flagship strategy is documented in `IVB_Model_Documentation.pdf` and `strategies/ivb_model/CLAUDE.md`.

Contents

1. [Project overview](#)
 2. [Running the app](#)
 3. [Folder structure](#)
 4. [Settings and data roots](#)
 5. [Asset info and auto parameters](#)
 6. [Data layer and transforms](#)
 7. [C++ order-book kernel](#)
 8. [Forex Factory calendar](#)
 9. [The strategy engine](#)
 10. [The modules \(windows\)](#)
 11. [Plugin contracts](#)
 12. [Volume profile algorithm \(peak-based\)](#)
 13. [End-to-end data flow](#)
 14. [Known limitations and roadmap](#)
 15. [What changed in this revision](#)
-

1. Project overview

A modular **intraday futures research platform** for 30+ instruments, built in Python as a native **PySide6 desktop app** (rebuilt from Streamlit in July 2026 with no logic changes). It converts raw Databento market data into enriched candle datasets, runs backtests through one shared **strategy engine**, sweeps parameter grids in the **Optimizer**, applies position sizing, runs Monte-Carlo stress tests, and labels market regimes. A C++ (pybind11) extension replays L3 order-book data, and a Forex Factory parser tags trades by news and holidays.

Extensibility (plugin drop): adding a data transform, strategy, position sizer, Monte-Carlo method, regime detector or quick script means dropping a file (or a package folder, for strategies) into the right folder. There is no registration: folders are scanned and modules loaded dynamically

(`importlib.util.spec_from_file_location + exec_module`), and the file name becomes the name shown in the UI. Every plugin category except Monte-Carlo methods can also be searched in extra folders added in Settings.

2. Running the app

```
python main.py
```

`main.py` is spawn-safe: nothing runs outside its `if __name__ == "__main__":` guard, because the Optimizer's worker processes re-import it. It starts `modules/app.py` (dark theme, settings) and opens the **main menu**, a card launcher. Every module opens in its own window; the same module can be opened several times and the windows are fully independent. Long-running work (transforms, backtests, grids, simulations) runs on worker threads, so windows never freeze, and every long job has a **Cancel** button. The gear icon on the main menu opens Settings.

Backend rule: inside every module, `backend/` is pure computation with no Qt imports (safe for process-pool workers and tests); `window.py` and the other UI files are the PySide6 front end. `tests/test_qt_smoke.py` fails if the Optimizer worker import chain pulls in PySide6.

3. Folder structure

<code>main.py</code>	entry point (spawn-safe)
<code>modules/</code>	
<code>app.py</code>	QApplication bootstrap
<code>engine/</code>	THE strategy engine (Qt-free): runner, day, cache, trade, timing – see section 9
<code>main_menu/</code>	launcher window
<code>common/backend/</code>	settings, asset_info, plugins, data_roots, trade_files, regime_join
<code>common/ui/</code>	theme, workers, widgets, params_form, settings dialog, additional_data, engine_progress, charts/
<code>common/trade_report/</code>	THE shared trade report (Backtester + Optimizer)
<code>data_formatter/</code>	Data Formatter window
<code>backtester/</code>	Backtester (backend/run.py, day_types.py)
<code>analytics/</code>	Analytics
<code>monte_carlo/</code>	Monte Carlo (+ methods/ plugin folder)
<code>optimizer/</code>	Optimizer (backend/ engine, param_space, metrics, combine/, ...; UI tabs Run / Explore / Combine)
<code>scripts/</code>	Scripts launcher
<code>regime_detector/</code>	Regime Detector
<code>strategies/</code>	strategy plugins (file or package)
<code>data_transforms/</code>	raw DBN -> parquet plugins
<code>position_sizing/</code>	fixed.py, kelly.py, risk_based.py
<code>regime_detectors/</code>	regime-detector plugins
<code>scripts/</code>	quick-script plugins
<code>forex_factory_scraper/</code>	FF calendar text -> ff_usd_events.parquet
<code>orderbook_replay_cpp/</code>	C++ (pybind11) L3 order-book replay kernel
<code>docs/</code>	sources of the generated guide and this document
<code>tests/</code>	pytest suite

Market data lives **outside the repository** in one or more **data roots** (default `D:/market_data`). Each data root is a full tree:

<code><root>/raw_dbn/{type}/{ASSET}/{dataset}/</code>	<code>*.dbn.zst</code> (immutable inputs)
<code><root>/parquet/{type}/{ASSET}/{dataset}/</code>	<code>YYYY-MM-DD.parquet</code> (working layer)
<code><root>/trades/{name}.parquet</code>	saved backtests (flat)
<code><root>/optimizations/{run}/</code>	optimizer runs (trades.parquet + meta.json)
<code><root>/regimes/{ASSET}/{run_name}/</code>	regime labels (daily parquets + meta.json)

```
<root>/news_and_holidays/ff_usd_events.parquet
<root>/temp/                                hand-over files (Go to Analytics / Monte Carlo)
```

Structural rules

- Raw DBN files are immutable inputs; the app never writes to them.
- Parquet is the working layer; reprocess by re-running a transform.
- The three-level hierarchy **type / ASSET / dataset** is enforced everywhere. Asset folders are UPPERCASE tickers (ES, NQ, GC).
- One parquet file per trading day, named **YYYY-MM-DD.parquet** (the RTH date; the file holds the Globex session 18:00 → 17:00 New York).
- Every candle parquet is indexed by a tz-aware **DatetimeIndex** in **America/New_York**. OHLC is float64.
- Pickers show the union of all data roots; **outputs are written to the root the input came from**.

4. Settings and data roots

settings.json (repo root, git-ignored, created on first start) holds:

Key	Meaning
version	schema version (1)
extra_plugin_dirs	extra folders per plugin category: strategies , data_transforms , position_sizing , scripts , regime_detectors (each also searches its in-repo default folder)
data_roots	list of data roots (section 3)
cache_gb	the engine's RAM day-cache budget in GB for the app process (default 4.0, range 0.5–512). Shown in Settings as "Engine day cache". Used by the Backtester and by serial Optimizer runs (section 9.4)
ui_prefs	per-module UI preferences, e.g. the trade report's section order and visibility

5. Asset info and auto parameters

modules/common/backend/asset_info.py holds the **one ASSET_INFO** table of the app (the Streamlit app had four copies). It maps every ticker to **tick_size**, **ticks_per_point**, **dollars_per_tick**, **commissions_per_contract** and, for micro / nano contracts, **parent** (the full-size contract with the same underlying). Helpers built on **parent**: **root_asset**, **same_underlying** and **default_reference_asset** (used by the trade report's regime and market-exposure pickers, section 10.3).

AUTO_PARAMS = {"tick_size"} (replaces the old **HIDDEN_PARAMS**): parameters filled from the selected asset. They are **shown read-only** in the parameter forms (greyed out, updated when the asset changes), are never swept by the Optimizer, and the engine always injects them into **params**.

The Backtester converts **pnl_points** to ticks with **ticks_per_point**; Analytics and Monte Carlo take **dollars_per_tick** and commissions from the asset, which they read from the **first underscore token of the trades file name** (**ES...parquet** → ES).

Ticker	tick_size	ticks_per_point	\$ per tick	commission / side	parent
ES	0.25	4	12.5	2.88	
NQ	0.25	4	5	2.88	
RTY	0.1	10	5	2.88	
YM	1	1	5	2.88	
MES	0.25	4	1.25	0.95	ES
MNQ	0.25	4	0.5	0.95	NQ
NNQ	0.5	2	0.1	—	NQ
M2K	0.1	10	0.5	0.95	RTY
MYM	1	1	0.5	0.95	YM
ZN	0.015625	64	15.625	2.3	
ZB	0.03125	32	31.25	2.37	

Ticker	tick_size	ticks_per_point	\$ per tick	commission / side	parent
ZF	0.0078125	128	7.8125	2.15	
ZT	0.00390625	256	7.8125	2.15	
SR3	0.0025	400	6.25	2.1	
CL	0.01	100	10	3	
QM	0.025	40	12.5	2.7	
NG	0.001	1000	10	3.1	
RB	0.0001	10000	4.2	3	
HO	0.0001	10000	4.2	3	
GC	0.1	10	10	3.1	
MGC	0.1	10	1	1.2	GC
SI	0.005	200	25	3.1	
HG	0.0005	2000	12.5	3.1	
ZC	0.25	4	12.5	3.6	
ZS	0.25	4	12.5	3.6	
ZW	0.25	4	12.5	3.6	
6E	0.00005	20000	6.25	3.1	
6J	0.0000005	2000000	6.25	3.1	
6B	0.0001	10000	6.25	3.1	
6C	0.00005	20000	5	3.1	
BTC	5	0.2	25	8	

Strategies that hard-code RTH as 09:30–16:00 New York are wrong for CL (pit 09:00–14:30), rates, FX and metals; `ivb_model` has a `session_start` parameter, the other strategies don't yet.

6. Data layer and transforms

6.1 Raw data (Databento DBN)

Source files are `.dbn.zst` archives from Databento (dataset GLBX.MDP3). Schema families used:

- **Trades / top of book** (TRADES, MBP-1 / TBBO, OHLCV-1m): the 1-minute candle and big-trade transforms.
- **MBO** (L3, full order book: every add / cancel / modify / fill with an order id): the 1-second order-book transforms, through the C++ kernel.
- **DEFINITION + STATISTICS**: the options and settlement transforms.

6.2 Session construction

Most transforms read a **pair of consecutive day files** (the previous evening and the current day) to build one complete ~23-hour Globex session:

- load previous + current day, indexed by `ts_event` (UTC);
- trim the previous day to `[22:00 UTC, ...)` and the current day to `[..., 21:00 UTC)` — the Globex session, labelled in New York time 18:00 → 17:00;
- concatenate, sort, convert the index to America/New_York;
- pick the front-month contract by highest volume; flag a roll day when the back month exceeds 20% of the front month's volume (still processed on the front month).

6.3 The transforms (`data_transforms/`)

Every transform exposes `run_all(input_folder, output_folder, skip_existing, on_progress[, params])`; transforms that declare **PARAMS** get a parameter form in the Data Formatter and receive its values as `params`.

Transform	Input	Output (one parquet per day)
<code>1m_advanced.py</code>	TBBO / MBP-1 day pairs, one asset	enriched 1-minute candles (6.4); front month, trade date and roll flag in the parquet metadata
<code>1m_advanced_indicators.py</code>	<code>*_1m_advanced</code> parquet	29 indicator columns (6.5)
<code>1m_ohlcv_globex_mixed_assets.py</code>	monthly OHLCV-1m files holding all ~30 assets	plain OHLCV per asset per day, routed to each asset's folder
<code>1m_ohlcv_globex_single_asset.py</code>	daily OHLCV-1m files of one asset (laid out like <code>1m_advanced</code> 's input)	the same OHLCV candles for that asset (new, see 6.6)
<code>1s_mbo_full_book.py</code>	MBO day pairs	1-second candles + the full resting book per second (6.7)
<code>1s_mbo_cropped.py</code>	MBO day pairs	same schema, book cropped around the traded range (6.7)
<code>big_trades.py</code>	TRADES stream	large pre-RTH and RTH trades (size thresholds in <code>PARAMS</code>)
<code>options_5m.py</code>	DEFINITION + STATISTICS + TBBO (ES options)	5-minute option candles per contract over the Globex session
<code>options_oi_snapshot.py</code>	DEFINITION + STATISTICS (ES options)	end-of-day chain with open-interest snapshots and prior settlement
<code>settlement_reference.py</code>	daily STATISTICS files, one asset	daily settlement and session reference table

6.4 Enriched 1-minute candles — `1m_advanced.py`

Column	Type	Description
<code>open / high / low / close</code>	float64	OHLC of the trades in the 1-minute bar
<code>volume</code>	uint32	total contracts traded
<code>buy_volume / sell_volume</code>	int32	contracts on the buy / sell aggressor side
<code>volume_delta</code>	int32	<code>buy_volume - sell_volume</code>
<code>volume_delta_pct</code>	float64	<code>volume_delta / volume * 100</code> , bounded ± 100 ; zero-volume bars = 0.0
<code>tick_volume</code>	JSON str	<code>{price: [buy_qty, sell_qty]}</code> per price level in the bar
<code>passive_orders</code>	JSON str	<code>{price: [size, order_count]}</code> resting liquidity versus the bar open

Gap filling: zero-volume bars forward-fill OHLC from the prior close with volumes 0. The enriched columns exist only for **ES and NQ** (plus the micros and NNQ where downloaded); other assets have OHLCV only.

6.5 Indicators — `1m_advanced_indicators.py`

Reads `*_1m_advanced` parquet, writes one indicators parquet per day (same file name) with 29 columns:

VWAP	Anchor	Notes
<code>vwap_bar_globex</code>	18:00 NY (first bar)	typical price (H+L+C)/3 weighted by bar volume
<code>vwap_bar_rth</code>	09:30 NY	NaN before 09:30 and after 17:00
<code>vwap_tick_globex</code>	18:00 NY	VWAP of the <code>tick_volume</code> price distribution
<code>vwap_tick_rth</code>	09:30 NY	NaN before 09:30 and after 17:00

Each VWAP carries $\pm 1/2/3\sigma$ bands (`_std1_up ... _std3_dn`, 7 columns each = 28), plus `cumulative_delta` = `cumsum(buy_volume - sell_volume)`, anchored at 18:00 NY and reset per file.

6.6 1-minute OHLCV — mixed and single asset

`1m_ohlcw_globex_mixed_assets.py` demultiplexes **monthly** all-asset OHLCV-1m files into one parquet per asset per day. `1m_ohlcw_globex_single_asset.py` (new in this revision) produces **exactly the same candles and file format** for one asset from its **daily** OHLCV-1m files: like `1m_advanced.py`, it splices each day with the previous day's file to build the 18:00 → 17:00 session, and it has the same customization as the mixed-asset transform (it is not hard-coded to an asset). Both write `open/high/low/close` (float64) and `volume` (int64) on the full 1380-bar grid, OHLC forward-filled and volume 0 on empty minutes.

6.7 1-second L3 book — `1s_mbo_full_book.py` and `1s_mbo_cropped.py`

One parquet per Globex session on a 1-second grid (end-of-second book snapshots). Trade fields are vectorized in pandas; book fields come from one sequential L3 replay in the C++ kernel (section 7).

Column	Type	Description
<code>open / high / low / close</code>	float64	trade OHLC per second
<code>volume / buy_volume / sell_volume</code>	int	volume and aggressor split per second
<code>best_bid / best_ask</code>	float64	top of book at the end of the second
<code>aggressor_volume</code>	JSON str	{ <code>price</code> : [<code>buy</code> , <code>sell</code>]} traded volume by price that second
<code>bid_depth / ask_depth</code>	JSON str	{ <code>price</code> : <code>size</code> } resting book per side

- **full_book** writes the entire resting book each second (pure-Python fallback when the extension is missing).
- **cropped** keeps only levels within $\pm N_TICKS$ of the second's traded high/low plus far "big" orders ($size \geq BIG_ORDER_MULT \times$ a rolling near-book median over `BASELINE_WINDOW_MIN` minutes); defaults 100 / 2.0 / 30. Requires the extension.

Size warning: the book JSON makes these files large in memory: one ES day with **all** columns is ~225 MB in RAM (about 20 MB on disk), so 136 days need ~30 GB. Strategies should declare only the columns they need (section 9.1); OHLC only is ~3 MB per day.

7. C++ order-book kernel

`orderbook_replay_cpp/` (pybind11 / setuptools; it replaced the Rust `heatmap_rs` in July 2026, output byte-identical) replays L3 (MBO) events sequentially, maintaining an aggregated depth ladder and an order-id map, and emits one end-of-second snapshot per active second:

```
replay_full(acode, scode, price_i, size, oid, sec)
-> (secs, best_bid, best_ask, bid_json, ask_json)      # full book
replay_cropped(acode, scode, price_i, size, oid, sec,
               n_ticks, tick_i, mult, window_sec, trade_sec, trade_lo, trade_hi)
-> (secs, best_bid, best_ask, bid_json, ask_json)      # cropped + big far orders
```

Inputs are pre-encoded numpy arrays: action code (A=0 C=1 M=2 F=3 R=4 T=5), side code (B=0 A=1 N=2), `price_i = round(price * 1e9)`, `sec` = epoch second. Build with MSVC (Visual Studio 2022):
`./venv/Scripts/python.exe -m pip install ./orderbook_replay_cpp.`

8. Forex Factory calendar

`forex_factory_scraper/ff_parser.py` parses plain-text Forex Factory USD calendar exports and writes `ff_usd_events.parquet` (`date` Date, `time` "HH:MM" or "All Day", `event`, `impact` = "red" for high impact or "grey" for holidays / bank closures). The app reads it from `<data root>/news_and_holidays/ff_usd_events.parquet` (the dataset's own root first, then any configured root). The Backtester tags every trade with `day_type` (holiday / FOMC / CPI / NFP / PPI / other high impact / normal) and the Optimizer with `day_bucket`; both feed the reports' day-type filters and breakdowns.

9. The strategy engine

`modules/engine/` (new in September 2026, extended in this revision) is the one place that runs strategies, for the Backtester and the Optimizer alike. It is pure Python with no Qt, so Optimizer worker processes import it. A strategy only **declares its data** and writes **per-day logic**; the engine owns everything else: the day loop, file reads, background read-ahead, the RAM cache, lookback to earlier days, timing, progress reporting and the output table.

File	Contents
<code>runner.py</code>	<code>run_strategy</code> (the day loop), <code>read_day_file</code> , <code>strategy_spec</code> , <code>additional_slots</code> , <code>day_files</code> , <code>estimate_run_memory</code> , <code>skipped_warnings</code> , <code>RunResult</code> , <code>EngineError</code>
<code>day.py</code>	<code>Day</code> / <code>DayData</code> — what <code>process_day</code> receives
<code>cache.py</code>	<code>DayCache</code> , the process-wide RAM cache (<code>get_cache</code> , <code>set_budget_gb</code> , <code>clear_cache</code> , <code>estimate_nbytes</code>)
<code>trade.py</code>	<code>Trade</code> and the 12 <code>OUTPUT_COLUMNS</code> (<code>trades_to_frame</code>)
<code>timing.py</code>	<code>timed</code> — the per-run timing table

9.1 The strategy contract

```
PARAMS      = {...}                                # defaults (+ PARAM_SECTIONS, PARAMS_OPTIONS)
DATA        = {"main": ["open", "high", "low", "close"], # columns per data slot, or "*"
               "indicators": ["vwap_bar_rth"]}          # extra keys = additional data
PREPARE_PARAMS = ["session_start"]                    # optional
def prepare_day(date, data, params): ...               # optional, param-independent, cached
def process_day(day, params) -> Trade | list[Trade] | None
```

- **DATA:** "main" is the dataset picked at the top of the window. Every other key is an **additional data slot**: a required "Additional data" row in the Backtester and the Optimizer, auto-picked by folder name (slot `indicators` → `ES_1m_indicators`). Only the declared columns are read; a declared column missing from a file is an error naming the column and file; a day whose additional file is missing is **skipped** and reported in a loud warning. (The old `indicators_folder` / `indicators_dataset` parameters are gone.)
- `prepare_day(date, data, params)`: work that does not depend on parameters (except those in `PREPARE_PARAMS`, which become part of its cache key). Its result is cached; returning `None` skips the day.
- `process_day(day, params)`: the trading logic for one day. `day.date`, `day.data[slot]` (DataFrames of the declared columns), `day.prepared`, `day.previous` and `day.previous_days(n)` (whole earlier days, every slot, crossing the run's start date), `day.missing`.
- **Days are independent:** `process_day` must not carry state from one day to the next (module variables), because a parallel Optimizer run processes a combination's days in several processes (9.6). Lookback goes through `day.previous`, which works across those chunks.
- **Trade** (one fixed class): `direction` ("long"/"short"), `entry_time`, `exit_time`, `entry_price`, `exit_price`, `exit_reason`, optional `sl`, `tp`, `trade_type` and `notes` (a flat dict, stored as a JSON string). Prices are in **points**.
- **Output** (always these 12 columns): `date`, `direction`, `trade_type`, `entry_time`, `exit_time`, `entry_price`, `exit_price`, `sl`, `tp`, `exit_reason`, `pnl_points`, `notes`. The engine computes `date` and `pnl_points` (exit – entry for longs, entry – exit for shorts). Strategies never compute ticks.
- `params = {**PARAMS, **UI values, "tick_size": <from ASSET_INFO>}`.

9.2 The day loop

1. Validate the strategy and the folders of every data slot.
2. List the main dataset's day files in [start, end] (oldest first).
3. Decide the read-ahead from the first day's parquet footers: normal days get **2 background read threads and 4 days of read-ahead**; "heavy" days (declared columns over 32 MB in RAM, e.g. full-book MBO) get **4 threads and 6 days**, because their decoding runs mostly outside Python's GIL (for small files extra threads only compete with the strategy).
4. Per day: wait for its reads if needed, skip it if an additional slot has no file, get or build the prepared day, call `process_day`, collect the trades.

5. Build the 12-column table, the summary, the warnings and the timing table.

Reads: `read_day_file` reads a day file directly with pyarrow (`ParquetFile.read(columns) → to_pandas`). It returns frames byte-identical to `pd.read_parquet` (verified on 8,508 reads across every dataset) but about twice as fast for small files, because it skips pandas' per-call overhead. The engine also memoizes pyarrow's time-zone lookup: without `pytz` installed, pyarrow retried a failing `import pytz` (a full `sys.path` scan) on every file read.

9.3 Results: `RunResult`

`trades` (the 12 columns), `warnings`, `days_in_range`, `days_run`, `skipped` (`{slot: [dates]}`), `files_read` (disk reads), `days_prepared` (`prepare_day` calls), `cache_rejected` / `cache_note` (the cache was too small), `elapsed`, `summary` (one-paragraph text), `timing` (the timing table) and `rows` (the `(date, Trade)` pairs behind `trades`).

9.4 The RAM cache

Day data "floats" in RAM between runs, because reading files is the slow part. `DayCache` is one least-recently-used store with one byte budget, holding:

- **raw** entries: the declared columns of one day file, keyed by (path, modification time, size, columns), shared by every strategy reading the same file and columns;
- **prepared** entries: a strategy's `prepare_day` result, keyed by (strategy, date, the identities of every slot file, the prepare parameters).

Budget: **Settings** → **Engine day cache** (`cache_gb`, default 4 GB) for the app process (Backtester and serial Optimizer runs); for parallel Optimizer runs the **Memory budget** of the Run tab, split evenly across the workers.

Eviction is **scan-resistant**: first raw frames whose day is already prepared and cached, then entries the current run hasn't used, and if everything left is in use by the current run, the **new** entry is not kept. A cache smaller than the run therefore keeps the run's first days for the next run instead of cycling everything out. When that happens, the run reports a yellow note ("The RAM cache (X GB) was too small for this run ..."), also stored as `cache_note` in an Optimizer run's `meta.json`.

Sizes are measured exactly for DataFrames (equal to pandas' deep memory usage, computed ~40× faster from the column arrays) and estimated for prepared objects (numpy arrays including their object header, containers, strings; or the object's own `__cache_nbytes__()`). A prepared day built during a run is **re-measured once after its first process_day**, so strategies that fill lazy caches (ivb_model parses its JSON then) are counted at their real size.

File changes are detected automatically (modification time and size are in the keys); **code changes are not**: after editing a `prepare_day`, press **Free cached data** (Backtester and Optimizer Run tab).

9.5 Progress and timing

At **every step** the engine calls `on_progress(day_i, n_days, text)` with four lines:

```
Day 123 of 372 · 2025-10-09 – preparing the day (prepare_day)
Reading ahead on 2 background threads: 2025-10-10 ... 2025-10-15
So far: 57 trades · 122 days processed · files read from disk: main 126, indicators 126 · files from RAM: 0 ·
    prepared days: 123 built, 0 from RAM
RAM cache 0.11 / 4.00 GB · elapsed 2.1 s · about 4.3 s left
```

Line 1 (the current step) is exact at every call: waiting for the disk, preparing the day, prepared day found in RAM, running the strategy, or skipped (with the reason), plus the setup and wrap-up steps. Lines 2–4 are rebuilt at most every 0.1 s. Raising an exception inside the callback cancels the run (the app's Cancel buttons). UIs must store the latest call and paint on a timer.

Every stage is timed into **one table** per run (`RunResult.timing`, printed to the console and shown in the Backtester): `engine:setup`, `engine:list_days`, `engine:read_plan`, `engine:schedule`, `engine:file_stat`, `bg io:read:<slot>` and `bg cache:store` (on the read-ahead threads, overlapping the main thread), `io:wait_for_read`, `cache:lookup` / `cache:store` / `cache:remeasure`, `day:prepare`, `day:process` (with the strategy's own `timed` sections nested inside), `engine:collect_trades`, `engine:progress` and `build_output_df`.

9.6 Engine performance (October 2026 measurements)

Backtester, all days of each dataset, best of 3 interleaved rounds; cold = empty cache (files read from disk), warm = everything in RAM:

Run	Before cold	After cold	Change	Before warm	After warm
ivb_model, ES 1m advanced	5.48 s	4.52 s	-17%	0.61 s	0.59 s
ivb_model, NQ 1m advanced	12.33 s	11.04 s	-10%	0.90 s	0.88 s
vwap_trend, NQ 1m advanced	2.22 s	1.31 s	-41%	0.18 s	0.16 s
orb, ES 1m advanced	0.87 s	0.46 s	-47%	0.03 s	0.03 s
example_prev_day_levels, ES	2.17 s	1.28 s	-41%	0.38 s	0.31 s
first-hour breakout, ES MBO (OHLC)	0.73 s	0.56 s	-23%	0.31 s	0.31 s
orb, NQ MBO	0.31 s	0.19 s	-37%	0.01 s	0.01 s
ES MBO, all columns	6.06 s	4.76 s	-22%	0.01 s	0.02 s
NQ MBO, all columns	7.15 s	5.69 s	-20%	0.01 s	0.01 s

Optimizer, 16 combinations (8 for the full-book grid), every worker starting empty; total time and (in brackets) when the first combination was complete:

Grid	Before	After
ivb_model ES, 8 workers	13.9 s (12.3 s)	4.4 s (2.5 s)
ivb_model NQ, 8 workers	35.9 s (32.3 s)	9.8 s (4.9 s)
first-hour breakout, ES MBO, 8 workers	3.8 s (2.6 s)	2.9 s (1.7 s)
ES MBO full book, 8 workers, 40 GB	46.8 s (45.6 s)	7.9 s (7.1 s)
ivb_model ES, serial	14.5 s	13.1 s
ES MBO full book, serial	7.2 s	5.0 s

Every strategy produced byte-identical trades before and after (13 reference configurations, and every grid old/new and serial/parallel).

10. The modules (windows)

10.1 Data Formatter

Pick an input dataset (raw DBN or parquet) with cascading type / asset / dataset pickers, pick a transform (its **PARAMS** form appears), name the output folder and run. **skip_existing** makes re-runs incremental; progress streams to a log.

10.2 Backtester

Pick type / asset / dataset (the main data), the strategy, the dates, check the **Additional data** rows (one per extra **DATA** slot, auto-picked) and the parameters (**tick_size** read-only, following the asset), then **Run**:

- a **progress panel** shows the engine's live four lines (9.5) with a **day i / n** bar; **Cancel** stops after the current step;
- when the run ends, the panel shows the engine's summary (days, trades, files read from disk vs from RAM, prepared days built vs reused, cache fill) and a collapsible **Engine timing of the last run** table;
- data warnings (skipped days, cache too small) appear in a yellow banner;
- **Free cached data** empties the engine cache (after editing a **prepare_day**, or to release RAM).

The window adds **ticks** = **pnl_points** × **ticks_per_point** and **cumulative_ticks**, tags **day_type**, and shows the trade report (10.3). **Save Trades** writes the filtered trades to `<root>/trades/{dataset}_{strategy}_{start}_{end}.parquet` (12 columns + **ticks** + **cumulative_ticks**, active filters in the parquet metadata); **Go to Analytics / Monte Carlo** hand the trades over through `<root>/temp/`.

10.3 The trade report (shared)

`modules/common/trade_report/` is **one** implementation used by the Backtester and both Optimizer drill-downs (cell and Combine). Its sections form an ordered stack whose order and visibility (always visible / dropdown / hidden) are set with the gear and saved in `ui_prefs`: trade-type filter, day-type filter, Performance metrics, **Equity Curve**, chart view settings, trade detail, **Strategy Performance by Regime**, entry breakdown, news and holiday exposure, **Market Exposure (α/β regression)**, exit breakdown, R:R distribution, Trades & Notes, and Save / Go to.

Changes in this revision:

- **Drawdown chart** joined under the equity curve (resizable splitter): the drawdown in ticks from the running peak, which **starts at 0** (so a losing start counts as drawdown), drawn in red with a lighter red fill down from the zero line. Hovering shows the drawdown in ticks; a marker shows the maximum drawdown; the crosshair spans both charts; clicking a trade highlights it on both. The **Max DD** metric tile uses the same start-at-0 definition.
- **Regime and market-exposure asset**: both sections have a dropdown to pick the asset / folder the regimes or the benchmark returns come from (the checkbox stays). It defaults to the backtest's own asset; when that asset has none, it falls back to the parent contract from `ASSET_INFO` (MES → ES, MNQ / NNQ → NQ). The choice is remembered only for the window.

Regime labels are always joined **as of the trade's entry time**, never by date

(`modules/common/backend/regime_join.py`).

10.4 Optimizer

Three tabs: **Run** (configure and launch a grid), **Explore** (heatmaps of any metric, sliders, cell drill-down with the full trade report, train/test split at the median trade day) and **Combine** (pick a small, diverse set of entry-variant trade streams from saved runs and evaluate them merged under a one-position-at-a-time rule, without re-running backtests).

The Run tab sweeps up to 4 parameters (X axis, Y axis, two sliders): int/float over min/max/step, str over a value list, bool over [False, True], dropdowns over a subset of their choices, bit-flag groups over a bitstring list.

`tick_size` can't be swept. More than 2,000 combinations needs a confirmation.

Execution:

- **1 worker (serial)**: every combination is one engine run in the app process, sharing the app's cache with the Backtester, so data already in RAM is reused. The log shows one line per combination (with where its data came from) and, during a slow combination, one line a second with the engine's current step.
- **N workers (parallel) — changed in this revision: the dates are split, not the combinations**. The date range is cut into N contiguous chunks of days, one per worker (each worker is its own single-process pool). Every worker runs **every** combination on **its own** chunk: each day file is read and prepared once, by one worker, and every later combination runs from that worker's RAM. Even a single combination runs N× in parallel. The workers return their (`date`, `Trade`) rows and the parent joins each combination's chunks in date order into **exactly** the table a serial run builds. Before, every worker ran a share of the combinations over all the days, so all of them loaded the whole dataset at once and the first round took as long as a cold backtest (see the table in 9.6). The log shows a start line, one "worker k/N (dates, days) loaded its data in X s" line per worker, and one line per finished combination.
- **Memory budget** is the total for all workers' caches, split evenly — each worker only holds its 1/N share of the days, so the data is never duplicated. The tab shows an estimate from the strategy's declared columns and the parquet footers (`estimate_run_memory`): the day data in RAM, the share per worker, the worker processes' own memory (~0.2 GB each), and a warning when the budget is smaller than the data.
- Cancel stops after the in-flight part on each worker.

A saved run, `<root>/optimizations/{run}/`, holds `trades.parquet` (every combination's trades with the swept parameters as extra columns, plus `pnl_ticks` and `day_bucket`) and `meta.json`: `strategy`, `dataset`, `ticker`, `tick_size`, `ticks_per_point`, `start_date`, `end_date`, `axes`, `fixed_params`, `min_trades_default`, `be_band_ticks`, `event_keywords`, `ff_events_found`, `split_date`, `n_combos`, `n_trades`, `created_at`, `additional_data` (`{slot: "type/asset/dataset"}`), `data_warnings` and `cache_note`.

10.5 Analytics

Load saved trade files, apply a position sizer, and compare several sized instances (trades file + sizer + label).

`dollars_per_tick` comes from the file name's asset; costs from `ASSET_INFO` commissions. Reports total P&L, final equity, max drawdown (\$ and %), Sharpe (annualized, daily, $\sqrt{252}$), win rate, profit factor and skipped trades.

10.6 Monte Carlo

Resample the trade history into thousands of equity paths. Methods (`modules/monte_carlo/methods/`):

- **bootstrap** — resamples trades with replacement (edge uncertainty);
- **prop_firm** — stopping-time simulation of a prop-firm challenge, payout and the combined path (trailing end-of-day floor, daily loss limits, size clamps, withdrawal caps); `PROP_FIRM = True` swaps in its own panel;
- **regime_switching** — a Markov regime-switching simulation (not MCMC): a transition matrix from all days, trade outcomes drawn per regime, equity indexed by trading day; `REGIME_PANEL = True` swaps in its own panel.

Results: fan chart ($1/2/3\sigma$ bands, sampled, featured and median paths) and a statistics table.

10.7 Regime Detector

Runs a regime-detector plugin over a parquet dataset and writes per-snapshot labels, one file per day, to

`<root>/regimes/{ASSET}/{run}/` (+ `meta.json`). The runner enforces the strict-before snapshot rule and a never-stale lookback. Explore tab: regime chart over the candles.

10.8 Scripts

Lists quick one-off scripts from `scripts/`. A script with `# app: streamlit` (or `STREAMLIT = True`) in its first 30 lines runs as `streamlit run` in a dedicated private browser profile; others run as `python -u` with output in the window's console. The working directory is the script's own folder.

11. Plugin contracts

11.1 Transform

```
PARAMS = {...}                                # optional: rendered as a form
def run_all(input_folder, output_folder, skip_existing, on_progress, params=None) -> None: ...
# on_progress(current, total, message); raising inside it cancels
```

11.2 Strategy

Section 9.1, and in full detail `STRATEGY_GUIDE.md`. The old contract `run(folder_path, start_date, end_date, params) -> DataFrame, PARAM_SPACE` and `HIDDEN_PARAMS` no longer exist. A strategy may be a single file or a package folder whose `__init__.py` exposes the contract (`strategies/ivb_model/`, `strategies/orb/`, `strategies/vwap_trend/`, `strategies/example_prev_day_levels/`, `strategies/delta_pct_signal/` — a one-minute `volume_delta_pct` imbalance signal with time exits only, used to measure momentum vs reversal).

11.3 Position sizer

```
PARAMS = {"account_size": 100000.0, ...}
def apply(trades: pd.DataFrame, params: dict) -> pd.DataFrame: ...
# returns a copy + size, trade_pnl (ticks * $/tick * size), equity
# trades whose size rounds to 0 are skipped -> result.attrs["skipped_trades"]
```

11.4 Monte-Carlo method

```
PARAMS = {"n_paths": 1000, "seed": 42, ...}
def run(trades, sizer_module, sizer_params, params) -> dict: ...
# the dict includes equity_matrix (n_paths, n_steps + 1), n_trades, method
```

11.5 Regime detector

```
PARAMS = {"rth_start": ..., "rth_end": ..., "snapshot_minutes": ..., "lookback_days": ..., ...}
REGIME_STATES = [...]; COLUMN_TIERS = {...}; SCRIPT_VERSION = "..."
def run_all(input_folder, output_folder, skip_existing, on_progress, params) -> None: ...
# import the helper as: from modules.regime_detector.backend.runner import RegimeContext, SHARED_PARAMS
```

11.6 Parameter declarations (all plugins)

PARAMS defaults choose the widget: int/float → spin box, str → text, bool → checkbox. **PARAMS_OPTIONS** = {param: [choices]}: a default in the list → dropdown; a '0'/'1' bitstring default with one character per option → a named checkbox group returning the bitstring. **PARAM_SECTIONS** groups the form. The full rules are the docstring of `modules/common/ui/params_form.py`.

12. Volume profile algorithm (peak-based)

Used by `ivb_model`; tighter value areas than a single max-volume POC on bimodal profiles:

- aggregate **tick_volume** across the range into {price: total_volume};
- smooth with a 3-tick rolling average and find local maxima;
- cluster peaks within 4 ticks and keep the top 5 by smoothed volume;
- refine each peak to the highest raw-volume tick within ± 3 (the POC candidate);
- expand each candidate outward (higher adjacent side first) until 70% of the volume is captured → VAH / VAL;
- pick the candidate with the tightest VAH–VAL range and refine its POC to the highest raw tick inside it.

13. End-to-end data flow

```
raw_dbn/{type}/{ASSET}/{dataset}/ (Databento .dbn.zst)
| Data Formatter + transform
v
parquet/{type}/{ASSET}/{dataset}/YYYY-MM-DD.parquet
|-- Backtester + strategy (engine) ----> trades/{name}.parquet (+ day_type)
|-- Optimizer + strategy grid (engine) -> optimizations/{run}/ -> Combine ->
    optimizations/{container}/_combined/{run}/
|-- Regime Detector + detector -----> regimes/{ASSET}/{run}/ -> report regime section
trades/ --> Analytics + sizer --> sized equity curve + $ metrics
trades/ --> Monte Carlo + sizer --> equity_matrix -> fan chart + statistics
```

Data passes between stages as **files on disk** (parquet) and within a stage as DataFrames; the contract between stages is the parquet column schema.

14. Known limitations and roadmap

- **Session times:** RTH is hard-coded 09:30–16:00 New York in most strategies (`ivb_model` has `session_start`) — wrong for CL, rates, FX and metals.
- **Enriched data coverage:** **tick_volume** / **passive_orders** only for ES / NQ (and their micros / nanos where downloaded); other assets are OHLCV only.
- **Tick volume:** multi-level fills are recorded only at the reported price.
- **Contract rolls:** front month by volume; cross-roll backtests have a one-day discontinuity (no back-adjustment).
- **Bootstrap Monte Carlo** assumes independent trades (the regime-switching method addresses volatility clustering).
- **Parallel Optimizer workers start empty on every run** (new processes); the first combination loads each worker's chunk from disk.
- **The engine cache does not detect code changes** in `prepare_day` (press Free cached data).

Item	Status
pnl_points-only strategy output, ticks derived from ASSET_INFO	DONE
One ASSET_INFO + AUTO_PARAMS (read-only tick_size)	DONE

Item	Status
Strategy engine: engine-owned day loop, Trade class, data slots, RAM cache	DONE
Engine timing table + live progress; faster reads	DONE
Optimizer parallel runs split by dates (each day loaded once)	DONE
Drawdown chart; regime / exposure asset picker	DONE
Peak-based volume profile	DONE
MBO 1-second book transforms + C++ kernel	DONE
Forex Factory news / holiday tagging	DONE
Trailing stop (ivb_model <code>vwap_trailing_risk</code>)	DONE
Prop-firm and regime-switching Monte Carlo	DONE
Per-asset session times for every strategy	TODO
Walk-forward validation (needs multi-year enriched data)	TODO
Keep Optimizer workers alive between runs (warm start)	TODO (optional)

15. What changed in this revision

Compared with the June 2026 edition (which still described the Streamlit app):

- The app is a **PySide6 desktop app**; sections 2, 3 and 10 describe the windows.
- **Data roots** outside the repo and `settings.json` (section 4), including the new `cache_gb` engine cache budget.
- **One ASSET_INFO** in `modules/common/backend/asset_info.py` with `parent` links, commissions and NNQ; `AUTO_PARAMS` replaces `HIDDEN_PARAMS`.
- **New transform** `1m_ohlc_v_globex_single_asset.py` (6.6).
- The Rust `heatmap_rs` kernel is now the **C++** `orderbook_replay_cpp` (7).
- **The strategy engine** (9) replaces `strategy.run(...)`: `DATA` slots, `prepare_day` / `process_day`, the `Trade` class and the fixed 12 output columns, the RAM cache (scan-resistant, re-measured prepared days), lookback, pyarrow-direct reads, adaptive read-ahead, the step-by-step progress text and the timing table.
- **Backtester**: progress panel, Cancel, engine summary and timing table, Free cached data, Additional data rows, read-only `tick_size` (10.2).
- **Trade report**: drawdown chart and start-at-0 Max DD; asset pickers for the regime and market-exposure sections (10.3).
- **Optimizer**: parallel runs split the dates across workers, the strategy-aware memory estimate, richer progress log, `additional_data` / `data_warnings` / `cache_note` in `meta.json` (10.4).
- Plugin contracts updated (11); measured performance (9.6).